

CachyOS — Full Install + Gaming Setup (AMD)

This guide walks through the whole process, from downloading the ISO to a working gaming desktop on CachyOS (an Arch-based, performance-optimized distro). The setup targets an **AMD GPU** (the best case for Linux gaming — no proprietary drivers to manage).

Reference hardware this was done on:

- CPU: AMD Ryzen 5 7600X (6 cores / 12 threads)
- GPU: AMD Radeon RX 6800 (Navi21, RDNA2) + integrated Raphael graphics
- RAM: 32 GB
- SSD: Kingston KC3000 NVMe

If your friend has **Nvidia**, most of this guide still applies, but the GPU/driver section is different (proprietary drivers) — let me know and I'll add it.

0. Prerequisites

- USB stick (min. 8 GB, it gets fully overwritten)
 - **Always back up your data** — partitioning is safe, but a mistake wipes data. "Erase disk" deletes the entire drive including Windows.
 - Git projects pushed, important files in the cloud (Mega/Drive); game configs sync automatically via Steam Cloud
 - **Decide in advance:** full Linux (whole disk) or dual-boot (Linux alongside Windows)? See step 4 below.
-

1. Download the ISO + create bootable USB

1. Download the CachyOS ISO from cachyos.org (desktop edition, e.g. `cachyos-desktop-linux-*.iso`).
 2. Download **Rufus** from rufus.ie (if you're still on Windows).
 3. In Rufus:
 - Device → your USB stick
 - Boot selection → the ISO you downloaded
 - Click **START**
 - When asked about the mode, choose **DD Image mode** (CachyOS is a hybrid ISO)
 4. Wait for it to finish, close Rufus.
-

2. BIOS / boot

1. Reboot into BIOS (usually **(DEL)** or **(F2)** at startup).
2. **Disable Secure Boot.**
3. In the boot menu (**(F8)**/**(F11)**/**(F12)** depending on the board) pick the USB entry prefixed with **(UEFI:)** — important for a proper UEFI install.

If you're going dual-boot with Windows: while still in Windows, disable **Fast Startup** (Control Panel → Power Options → Choose what the power buttons do → Turn on fast startup = OFF). Otherwise Windows "hibernates" the filesystem and Linux can't safely mount shared/NTFS partitions (risk of data corruption).

3. Installation (Calamares)

After booting into the live environment, the **CachyOS Hello** window opens → click **Launch installer**.

Calamares walks you through these steps:

1. **Welcome** — language (English or your locale) → Next
2. **Location** — time zone (auto-detected) → Next
3. **Keyboard** — layout (most people stay on **US**) → Next
4. **Partitions** —  **the most important step.** Calamares usually offers these options: **a) Erase disk** (full Linux) — wipes the whole drive and installs only Linux. Simplest if you don't need Windows. **b) Install alongside** (dual-boot) — installs Linux next to the existing Windows in free space. Calamares shows a slider to split space between the two systems. The bootloader (GRUB/Limine) then shows a menu at startup to pick the OS. *Requires:* Windows already installed on the disk with free space available (or you shrank the C: partition beforehand via Disk Management in Windows). **c) Replace a partition** — overwrites just one specific partition with Linux, leaving the rest of the disk (including Windows) intact. Handy if you have a dedicated partition prepared for Linux. **d) Manual partitioning** — full control (your own **(/)**, **(/boot/efi)**, swap, **(/home)**). Only if you know what you're doing. Common settings (for most cases):

- Swap → **"swap (with hibernate)"**
- Filesystem → **Btrfs** (for snapshots — lets you roll back before a broken update)
- **Always verify the target disk/partition is the right one** (by model and size), so you don't accidentally overwrite the wrong drive.

Note on dual-boot: ideally install **Windows first** and Linux second — the Linux installer sets up GRUB, which auto-detects Windows. If you install Linux on a machine where Windows doesn't exist yet but you plan to add it later, it's more painful the other way around (Windows overwrites the bootloader and you have to repair it). **512 GB+** of disk is comfortable so both systems have room.

5. **Users** — name, user, password (you can tick "use same password for admin")
6. **Bootloader** — **Limine** (the default, actively developed and tested by CachyOS; GRUB is also fine if you want more documentation to fall back on)
7. **Summary** — verify the target disk → **Install**

When it finishes, tick **Restart now** → **Finish. Remove the USB during reboot** (when the monitor goes black) so it doesn't boot back into the live environment.

4. First steps after boot

After reboot it boots straight into KDE Plasma (SDDM login).

1. Verify internet (Wi-Fi/LAN).
2. **Update the system** first, always after a fresh install:

```
bash
sudo pacman -Syu
```

- If it asks `(Replace sdl2 with cachyos-extra-znver4/sdl2-compat?)` → confirm **Y/Enter**. This is normal — CachyOS has its own znver4-optimized packages. Confirm Y for anything from `(cachyos-*)` repos.
 - Btrfs + snapper automatically takes a **snapshot before and after the update** (root: 7 → 8) — if an update breaks something, you can roll back.
 - 3. In **CachyOS Hello** → **Install Apps** there's a built-in list for one-click installs (Steam, Discord, browsers...).
-

5. GPU / drivers (AMD)

On AMD you don't need to **install anything manually** — Mesa/RADV drivers are part of the system and the kernel. Driver updates = `(pacman -Syu)` (no separate "driver app" like Adrenalin on Windows).

Verify the GPU is running correctly:

```
bash
sudo pacman -S mesa-utils          # if glxinfo is missing
glxinfo | grep "OpenGL renderer"
```

Should show something like `(AMD Radeon RX 6800 (radeonsi, navi21, AC0))`.

Vulkan: uses **RADV** (Mesa, default, best for gaming). Ignore AMDVLK unless a specific game misbehaves.

6. Gaming — Steam + Proton

```
bash
sudo pacman -S steam
sudo pacman -S protonup-qt
```

Steps:

1. **Launch Steam**, sign in. Let it finish updating (it creates `(~/.local/share/Steam/compatibilitytools.d/)`). Your Dota config downloads via Steam Cloud automatically.
2. **Open ProtonUp-Qt** → "Install for" pick **Steam** → **Add version** → newest **GE-Proton** → **Install**.
 - If ProtonUp-Qt doesn't detect Steam (`(Installed compatibility tools: 0)`), close it, launch Steam first, then reopen ProtonUp-Qt.
3. In **Steam** → **Settings** → **Compatibility**:
 - Enable "Enable Steam Play for all other titles"
 - In the dropdown pick the GE-Proton you installed (it appears at the top above Proton Experimental; if not, restart Steam)
4. **Dota 2** — native Linux client, **no Proton needed**, just Install.
5. **Horror games** (Resident Evil, Silent Hill, etc.) — run automatically via GE-Proton.

Tip: before buying/installing a game, check [ProtonDB](#) — it shows compatibility and launch options (`(gamemoderun %command%)`, `(mangohud %command%)`, etc.).

7. Graphics control (AMD Adrenalin equivalents)

Adrenalin doesn't exist on Linux, but everything it does is covered:

Adrenalin feature	Linux tool
Driver updates	<code>(pacman -Syu)</code> (no separate app)
OC / undervolt / fan curves	CoreCtrl (or LACT)
FPS / temp / usage overlay	MangoHud (<code>(mangohud %command%)</code>)
Performance boost	GameMode (<code>(gamemoderun %command%)</code>)
Recording (ReLive)	OBS Studio (VAAPI/AMF HW encode)

CoreCtrl (fan curves, power limit, undervolt, per-game profiles):

```
bash
sudo pacman -S corectrl
```

⚠️ **Run it WITHOUT `sudo`** — just `corectrl`. It requests root rights internally via a polkit dialog (enter your normal password) only for operations that need it. Running with `sudo` throws cosmetic errors about `.cache/`/`/tmp/CoreCtrl.log`.

Sensors in the terminal (quick check):

```
bash

sudo pacman -S lm_sensors
sudo sensors-detect
sensors
```

8. RGB keyboard

For controlling RGB (e.g. HyperX Alloy Origins Core), use **OpenRGB**:

```
bash

sudo pacman -S openrgb
```

On CachyOS it often picks up the RGB hardware automatically. Launch it, pick the device, set color/effect.

9. Bonus tweaks (optional)

```
bash

sudo pacman -S gamemode lib32-gamemode mangohud
```

- **fstrim** (TRIM for SSD) — on CachyOS `fstrim.timer` is usually enabled by default, verify:

```
bash

systemctl status fstrim.timer
```

- CachyOS Hello → **Apps/Tweaks** — prebuilt gaming optimizations (kernel parameters). Look through them, but don't change anything blindly.
-

10. Dev tools (git, GitHub, editor, AUR)

git (and base build tools for compiling AUR packages):

```
bash
sudo pacman -S git base-devel
```

Set your git identity (once):

```
bash
git config --global user.name "Your Name"
git config --global user.email "you@example.com"
```

SSH key for GitHub — recommended over HTTPS. It avoids username/password (token) prompts entirely, which also sidesteps the credential-dialog freeze described in section 11:

```
bash
ssh-keygen -t ed25519 -C "you@example.com"      # press Enter through the prompt
cat ~/.ssh/id_ed25519.pub                      # copy this
```

Then paste it into GitHub → Settings → SSH and GPG keys → New SSH key. Clone with the SSH URL afterwards:

```
bash
git clone git@github.com:user/repo.git
```

AUR helper — CachyOS ships **paru** preinstalled (the AUR has packages not in the official repos). Usage:

```
bash
paru -S package-name
```

Editor — VS Code / Code - OSS:

```
bash
# Code - OSS (open-source build, from official repos; uses the Open VSX marketplace)
sudo pacman -S code

# OR the official Microsoft build (from AUR; full MS marketplace, includes Pylance)
paru -S visual-studio-code-bin
```

For a machine where you'll actually use the Claude Code extension, the official build (`visual-studio-code-bin`) is the smoother choice — see section 11 for why Code - OSS can freeze on sign-in.

11. FIX: Code - OSS / VS Code freezes on git clone / sign-in

Symptom: the whole Code - OSS freezes ("Not Responding") on a private git clone or while signing in (e.g. Claude Code sign-in). A public clone with no auth works fine, the network is fine.

Cause: an upstream **Electron bug** — native confirmation dialogs freeze the window on Arch-based distros with Code - OSS **1.123.0** / **1.124.0** (version 1.122.1 doesn't have the issue). Both of those operations open a native dialog (credential prompt / "open external website?"), and it's rendering that dialog that crashes. Tracked at: ([microsoft/vscode#321312](#)), ([electron/electron#51988](#)).

Fix — make VS Code draw its own in-window dialogs instead of native ones:

Command Palette (([Ctrl+Shift+P](#))) → **Preferences: Open User Settings (JSON)** → add:

```
json
{
  "window.dialogStyle": "custom"
}
```

Then **fully quit and relaunch** (not just reload window).

Or edit from the terminal (merge the line into the existing (`{ }`)):

```
~/.config/Code - OSS/User/settings.json
```

Once Electron is fixed in a future update, you can safely drop the setting (but you don't have to — custom dialogs work fine). Alternative — pin Code - OSS to 1.122.1 and add it to ([IgnorePkg](#)) in ([/etc/pacman.conf](#)) — also works, but holds updates back, so the custom-dialog setting is the cleaner option.

Quick command summary

bash

After install

```
sudo pacman -Syu
```

Dev tools

```
sudo pacman -S git base-devel
```

```
git config --global user.name "Your Name"
```

```
git config --global user.email "you@example.com"
```

```
ssh-keygen -t ed25519 -C "you@example.com" # add the .pub to GitHub
```

```
sudo pacman -S code # Code - OSS (or: paru -S visual
```

Gaming

```
sudo pacman -S steam protonup-qt
```

→ Steam login → ProtonUp-Qt: Add GE-Proton → Steam Settings: Steam Play for a

GPU check

```
sudo pacman -S mesa-utils
```

```
glxinfo | grep "OpenGL renderer"
```

Graphics control

```
sudo pacman -S corectrl # run WITHOUT sudo
```

```
sudo pacman -S lm_sensors && sudo sensors-detect
```

RGB

```
sudo pacman -S openrgb
```

Bonus

```
sudo pacman -S gamemode lib32-gamemode mangohud
```